

1. Fundamentos del lenguaje JavaScript

En el módulo de **Lenguajes de marcas y sistemas de gestión de información** seguramente ya tuviste una introducción a JavaScript. En este apartado haremos hincapié en los aspectos más avanzados.

JavaScript es un lenguaje de una gran complejidad. No se pretende explicar absolutamente todas sus características, sino solo hacer hincapié en las más importantes. En las referencias, tienes acceso a varios recursos que ofrecen una visión más amplia del lenguaje.

1.1. Entorno de ejecución de JavaScript

JavaScript es un lenguaje que puede ser utilizado en el navegador, y ejecutado en el cliente y en un servidor (NodeJS). El código puede ir definido de las siguientes formas:

- En una página HTML. Debe ir entre etiquetas `<script>` (no es necesario indicar el atributo `type="script"`).
- En un archivo `.js` independiente. Dicho archivo puede ser ejecutado en NodeJS y cargado por una página HTML mediante `<script src="RUTA_ARCHIVO.js">`.

1.2. Variables

Los nombres de variables en JavaScript deben contener únicamente letras, dígitos numéricos o los caracteres `$` y `_`. El primer carácter no puede ser un dígito. Además, existe un conjunto de **palabras reservadas** que no pueden ser utilizadas como nombre de variables, como **function**, **let**, **for**, **if**, etcétera.

Existen tres maneras de declarar variables en JavaScript:

- Mediante **var**. La variable quedará definida al ámbito de la **función** en la que esté declarada. Si no está declarada dentro de una función, la variable quedará definida en el **objeto global**, del que hablaremos a lo largo de la unidad.
- Mediante **let**. Es la **forma recomendada**. La variable quedará definida en el ámbito del **bloque** en el que esté declarada. Un bloque es un trozo de código limitado por `{ }`. En el caso de estar definida fuera de cualquier bloque, quedará definida en el objeto global o en el módulo donde esté definida.
- Mediante **const**. Queda definida de la misma manera que **let**, pero su valor no puede ser reasignado.

! IMPORTANTE

JavaScript busca las variables definidas a través de lo que se denomina **ámbito léxico**, o *lexical scope*. Cuando se hace referencia a una variable en el interior de una función (en el caso de **var**) o un bloque (en el caso de **let**), JavaScript busca primero si existe una variable local o un parámetro con el identificador de la variable buscada: si lo encuentra, lo utiliza; si no, busca en el siguiente nivel de función (o bloque), y así sucesivamente, hasta que llega a la raíz, que es el objeto global (programa convencional) o el módulo (si el código está definido en un módulo).

En JavaScript moderno, se tiende a utilizar sobre todo **let** para definir variables, ya que las variables creadas de esta manera siguen reglas más parecidas a la mayoría de los lenguajes de programación.

! IMPORTANTE

Utiliza **let** SIEMPRE que vayas a utilizar una variable en un **BUCLE**, tanto si es la variable que vas a iterar como si es una variable auxiliar que vas a declarar de manera local en dicho bucle.



EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo `DWEC_U02_A01_01.js`, con el código de ejemplo.

! IMPORTANTE

Siempre **declara** una variable mediante `let`, `const` o `var` antes de utilizarla. Si no lo haces, JavaScript creará una variable **automáticamente** en el **objeto global**, con consecuencias inesperadas. Otra solución consiste en utilizar el **modo estricto**, poniendo `"use strict"`; al principio del *script*: de esta manera, JavaScript generará un error si se trata de asignar un valor a una variable no declarada.

1.3. Operadores: aclaraciones

El operador suma, `+`, se utiliza también para **concatenar cadenas de caracteres**. Además, si uno de los operandos es una cadena, se realiza la **conversión a cadena** del otro operando:

```
console.log( '1' + 2 ); // "12"
console.log( 2 + '1' ); // "21"
```

Así, para utilizar el operador suma hay que **convertir las cadenas a números** previamente. Veremos cómo hacerlo en el siguiente punto.

Esto no ocurre con el **resto de operadores**, que realizan la **conversión a número**:

```
console.log( 6 - '2' ); // 4
console.log( '4' / '2' ); // 2
```

Los operadores también permiten realizar la operación de realizar **modificaciones sobre la misma variable**. Así:

```
let a = 5;
a += 5; // Es lo mismo que (a = a + 5)
a /= 3; // Es lo mismo que (a = a / 3)
```

1.4. Tipos de datos y conversiones

Los valores en JavaScript siempre son de un tipo de datos. Los tipos de datos son:

- **Tipos de datos primitivos:**
 - **number**. Para números, enteros o decimales.
 - **bigint**. Para números muy grandes.
 - **string**. Para cadenas de caracteres.
 - **boolean**. Verdadero (`true`) o falso (`false`).
 - **null**. Valor nulo.
 - **undefined**. Para variables declaradas no inicializadas (que no han recibido ningún valor).
 - **symbol**. Es un tipo de datos para crear **identificadores únicos e inmutables**, que se suelen utilizar para definir claves de objetos.
- **object**. Para **objetos** (incluidos *arrays* y funciones).

Dado que JavaScript es un lenguaje de tipado **dinámico**, las variables pueden albergar cualquier tipo de datos. Esto puede crear problemas en tiempo de ejecución, ya que al efectuar operaciones es posible que se produzcan **conversiones de tipos**. Consideremos el siguiente ejemplo:

```
let edad = prompt("Dime tu edad"); // El usuario introduce 40. Pero
"prompt" siempre devuelve un string, así que (edad = "40")
let edadMasDiez = edad + 10;
console.log(edadMasDiez); // "4010" (en lugar de 50)
// Se ha producido la conversión de tipo de 10 a "10", ya que el operador
"+" se ha comportado como "concatenar" en lugar de "sumar"
```

Las funciones y expresiones **matemáticas** realizan conversiones a **número** (con la excepción del operador **+**, que se convierte en el operador «concatenar», como acabamos de ver). Los casos especiales son:

Valor	Conversión
undefined	NaN
null	0
true / false	1 / 0
string	Si es una cadena vacía, 0; si se corresponde con un número, el número en cuestión; si no corresponde a un número válido, NaN .

Casos especiales

El valor **NaN** es un valor numérico que significa *not a number*.

Para **convertir a número** podemos utilizar varios métodos:

```
parseInt("3"); // 3
parseFloat("3.155"); // 3.155
Number("3"); // 3
+"3"; // 3 (Operador unario)

console.log(+ "3" + 4); // 7

// Ejemplo anterior ejecutado de manera correcta
let edad = prompt("Dime tu edad"); // El usuario introduce 40.
let edadMasDiez = Number(edad) + 10; // Convertimos "edad" a número
console.log(edadMasDiez); // 50
```

Es posible comprobar si una cadena de texto no corresponde a un número mediante la función **isNaN** (*is not a number*):

```
isNaN("5rt"); //true (no es un número válido)
isNaN("541"); // false
isNaN("285.32"); // false
isNaN("2e16"); // false
```

Las conversiones a **string** se realizan automáticamente. Por ejemplo, la función **alert** puede tomar como parámetro un número que convierte automáticamente a **string**. También se puede realizar la conversión explícita mediante **String(valor)**.

Las conversiones a **boolean** siguen las siguientes reglas:

Valor	Conversión
0, null, undefined, NaN, ""	false
Otro valor (incluida la cadena "0")	true

Conversiones a boolean

Los operadores de comparación de igualdad o «distinto de» tienen dos versiones: con **comprobación de tipo** (`===`, `!==`) o sin comprobación de tipo (`==`, `!=`, conversión de tipo automática). Así:

```
let (0 == false); // true
let ('' == false); // true

let (0 === false); // false (distinto tipo)
let ('' === false); // false (distinto tipo)
```

! IMPORTANTE

`null` y `undefined` son iguales `==` (igualdad no estricta) entre sí y distintos a cualquier otro valor.

Para terminar, una **recomendación**: si una variable puede almacenar `null` o `undefined`, compruébalo por separado mediante `==` (o `===`, si quieres diferenciar entre `null` y `undefined`) antes de realizar cualquier otra comparación.

1.5. Strings

Las cadenas de texto o **strings** pueden definirse mediante **comillas dobles** `"` o **sencillas** `'`. Si se desea utilizar comillas dentro de una cadena, habrá que utilizar un entrecomillado distinto. Por ejemplo:

```
let a = "Texto con 'comillas' dentro";
```

Existe una tercera manera de definir una cadena de texto: mediante **acento grave** (*backtick*) ```. Este modo tiene las siguientes ventajas:

- Permite crear cadenas de texto de **varias líneas**.
- Permite **incorporar expresiones** utilizando la estructura `${ }` dentro de la propia cadena. Esto evita tener que utilizar el operador de concatenar `+` y pensar en los espacios en blanco que se deben dejar para formatear adecuadamente textos que incorporen expresiones o datos almacenados en variables.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo `DWEC_U02_A01_02.js`, con el código de ejemplo.

! IMPORTANTE

El acento grave en el teclado español se muestra pulsando la tecla correspondiente del teclado (a la derecha de la «P») y pulsando la barra espaciadora a continuación.

1.6. Estructuras de control y bucles

JavaScript dispone de las estructuras clásicas de los lenguajes de programación: **condicionales** (**if**, **switch**) y **bucles** (**for**, **while**). Dichas estructuras han sido estudiadas en el primer curso del ciclo, por lo que se supone que son conocidas. Por ello, nos centraremos en estructuras algo más avanzadas.

La primera que veremos es el **operador ternario**, **?**. Este operador permite evaluar una condición de manera más concisa. Se utiliza para asignar un valor a una variable en función de una condición. Por ejemplo:

```
let mayorDe30;
let edad = prompt('¿Cuántos años tienes?');

if (edad > 30) {
    mayorDe30 = true;
} else {
    mayorDe30 = false;
}
```

El condicional puede reescribirse como:

```
let mayorDe30 = (edad > 30) ? true : false;
// Se evalúa la condición (edad > 30).
// Si es verdadera, se asigna a la variable "mayorDe30" el valor indicado
// tras '?' (true);
// Si es falsa, se asigna el valor indicado tras ':' (false).
```

La segunda estructura de interés es el bucle **for of**, que se utiliza para iterar sobre *arrays*:

```
let colores = ["azul", "rojo", "blanco"];

// Bucle "for" tradicional
for (let i=0; i<colores.length; i++) {
    console.log(colores[i]);
}

// Bucle "for of"
for (let color of colores) {
    // La variable "color" almacena el elemento del array
    // que se esté iterando
    // Equivalencia: color = colores[i]
    console.log(color);
}
```

! IMPORTANTE

Hay otra estructura, **for in**, que es parecida. La diferencia es que **for in** se utiliza para iterar sobre las propiedades de un objeto. Como en JavaScript los *arrays* son objetos, también funcionará, aunque no es recomendable usarla, ya que es más lenta y accede también a otras propiedades que no son los elementos del *array*.

1.7. Interacción básica

! IMPORTANTE

`alert`, `prompt` y `confirm` son parte de la API del navegador (dentro del objeto `window`). Por tanto, no están disponibles en el servidor (NodeJS).

Existen tres funciones básicas de interacción con el usuario: `alert`, `prompt` y `confirm`.

La función `alert` muestra un mensaje en un cuadro de diálogo mediante `alert("MENSAJE");`.

La función `prompt` muestra un cuadro de diálogo con un texto para que el usuario introduzca un valor, y devuelve dicho valor. La función tiene dos parámetros (el segundo es opcional):

```
let respuesta = prompt("TEXTO PARA MOSTRAR", "VALOR POR DEFECTO DEL CUADRO DE TEXTO (OPCIONAL)");  
// respuesta almacena el texto escrito por el usuario  
// OJO: siempre almacenará un string. Puede que sea necesario convertirlo a otro tipo de datos
```

! IMPORTANTE

`prompt` devuelve un `string` si el usuario escribe algo, y `null` si el usuario cancela la acción.

La función `confirm` muestra un cuadro de diálogo con un texto y dos botones: devuelve `true` si el usuario pulsa en Aceptar y `false` si pulsa en Cancelar. Su sintaxis es:

```
let acepta = confirm("¿Aceptas los términos?");  
// acepta almacena true si el usuario pulsa en Aceptar  
// y false en caso contrario
```

Por último, podemos citar la función `console.log("MENSAJE");`, que escribe un mensaje en la consola. Esta función sí que está disponible tanto en el navegador como en NodeJS.